Lab 3

Question

Use of Persona Prompt, Use of Cognitive Verifier Pattern, Question Refinement Pattern, Provide New Information and Ask Questions, Root Prompt

Code

1. Importing the required libraries

```
pip install langchain langchain-openai langchain-groq --quiet

from langchain_core.prompts import ChatPromptTemplate
from langchain_groq import ChatGroq
```

2. Master prompt to define the AI model

```
MASTER_PROMPT = ChatPromptTemplate.from_messages([
    ("system", """
You are a senior Coding AI and software engineering mentor.

PERSONA:
- Expert in clean code, performance optimization, and best practices
- Think critically and verify assumptions
- Explain like a professional code reviewer

ROOT BEHAVIOR RULES:
- Understand the task before responding
- Verify correctness and edge cases
- Do not hallucinate APIs or facts
- Prefer clarity, correctness, and maintainability

COGNITIVE VERIFIER PATTERN:
- Validate logic and recommendations
- Cross-check against standard Python/JavaScript best practices
- Ensure suggested changes truly improve the code
```

```
QUESTION REFINEMENT PATTERN:
- Identify missing or ambiguous information
- State assumptions explicitly
- Ask clarification questions only if they affect the outcome

PROVIDE NEW INFORMATION + ASK QUESTIONS:
- Introduce relevant best practices the user may not know
- Explain why they matter
- Ask thoughtful follow-up questions

REASONING:
- Reason internally step-by-step
- Do NOT reveal raw chain-of-thought
- Provide structured explanations only
"""),
    ("human", """
TASK:
Review the following JavaScript code for readability, performance, and best practices.

CONTEXT:
- Language: JavaScript
- Audience: Intermediate developers
- Focus: Code quality, efficiency, maintainability

Code:
{code}

OUTPUT REQUIREMENTS:
1. High-level assessment
2. Issues identified
3. Suggested improvements with justification
4. Refactored code (if appropriate)
5. New insights or best practices
6. Follow-up questions
""")
])
```

3. Importing the AI model

```
from google.colab import userdata

groq_api_key = userdata.get('GROQ_API_KEY')

llm = ChatGroq(
    model="openai/gpt-oss-120b",
    temperature=0.2,
    api_key=groq_api_key
)

chain = MASTER_PROMPT | llm
```

4. Example code

```
java_code = """
import java.util.ArrayList;
import java.util.List;

public class DuplicateFinder {

    public static List<Integer> findDuplicates(int[] arr) {
        List<Integer> duplicates = new ArrayList<>();

        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    duplicates.add(arr[i]);
                }
            }
        }
        return duplicates;
    }
}
"""
```

5. Response from the model

```
response = chain.invoke({
    "code": java_code
})

print(response.content)
```

Output

1. High-level assessment

Aspect	Current state	Verdict
Correctness	Finds *all* pairs of equal elements and adds the value for each pair.	Functionally works, but the result contains many repeated entries (e.g., for three identical numbers you get three copies).
Readability	Straight-forward nested loops, but variable names are generic (`i`, `j`) and the intent isn't documented.	Acceptable for a beginner, but can be improved with clearer naming and comments.
Performance	**O(n²)** time because of the double loop; **O(k)** extra space where *k* is the number of duplicate pairs.	Not scalable for large arrays.
Maintainability	Uses raw arrays and manual loops; any change (e.g., returning unique duplicates) requires rewriting the core logic.	Could be refactored to use the Java Collections API or Streams for easier future modifications.
API design	Returns a `List<Integer>` that may contain the same duplicate value many times.	Usually callers expect a *set* of duplicated values, not a multiset of pair-wise matches.

Overall the code is a decent first draft, but it suffers from three main problems:

1. **Quadratic time complexity** – unnecessary for a duplicate-detection task.
2. **Result duplication** – the same number can appear many times in the output.
3. **Lack of documentation & expressive naming** – makes the intent harder to grasp quickly.

---

2. Issues identified

#	Issue	Why it matters
1	**O(n²) algorithm** – two nested loops iterate over the whole array.	Becomes a bottleneck for arrays > 10^4 elements; memory-bound workloads suffer.

2	**Repeated duplicate entries** – each matching pair adds a new entry, so a value that appears *m* times yields *m·(m-1)/2* entries.	Callers typically need each duplicated value only once; extra work to de-duplicate later.
3	**No early exit / short-circuit** – even after a value is known to be duplicate, the inner loop continues checking it against the rest of the array.	Wastes CPU cycles.
4	**Using primitive `int[]`** – forces manual iteration; using a `Map` or `Set` would be more expressive.	Reduces readability and makes it harder to switch to other collection types.
5	**Missing Javadoc / comments** – the public method has no description of its contract (e.g., does it return unique duplicates?).	Future maintainers may misuse the method.
6	**Potential integer overflow** – not an issue here, but if the method were extended to count occurrences, using `int` counters could overflow for huge inputs.	Good to be aware when scaling.
7	**Naming** – `i`, `j` are fine for loops, but the method name `findDuplicates` could be more explicit about the return type (e.g., `findDuplicateValues`).	Improves discoverability via IDE auto-completion.

---

3. Suggested improvements with justification

Improvement	Explanation
Use a `HashMap<Integer, Integer>` (or `Int2IntOpenHashMap` from fastutil) to count occurrences	Counting each value in a single pass gives **O(n)** time and **O(u)** extra space, where *u* is the number of distinct elements.
Return a `Set<Integer>` (or `List<Integer>` with unique values)	Most callers only need to know *which* values are duplicated, not how many times they pair. Returning a `Set` guarantees uniqueness and conveys intent.
Add Javadoc	Documents the method contract, input expectations, and return semantics.
Rename method to `findDuplicateValues`	Clarifies that the method returns the *values* that appear more than once.
Consider a streaming API version	For Java 8+, `Arrays.stream(arr).boxed().collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))` is concise and expressive.
Validate input (null check)	Defensive programming prevents `NullPointerException` when the caller passes `null`.
Make the method generic (accept `int[]` or `Collection<Integer>`)	Improves reusability across different data sources.
If you truly need the *pairwise* duplicate list – keep the original algorithm but add a `Set<Integer>` guard to avoid adding the same pair twice.	Provides a clear alternative for the rare case where the exact pair count matters.

---

4. Refactored code (Java 8+)

Below is a clean, linear-time implementation that returns **unique duplicate values** as a `Set<Integer>`. It includes Javadoc, null-safety, and an optional overload that works with any `Collection<Integer>`.

```java
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/**
 * Utility class for finding duplicate values in integer collections.
 */
public final class DuplicateFinder {

    private DuplicateFinder() {
        // Prevent instantiation
    }

    /**
     * Returns a set containing all values that appear more than once in the given array.
     *
     * @param arr the input array; may be {@code null} or empty
     * @return an immutable set of duplicated values; never {@code null}
     */
    public static Set<Integer> findDuplicateValues(int[] arr) {
        if (arr == null || arr.length == 0) {
            return Collections.emptySet();
        }

        // Count occurrences in a single pass
        Map<Integer, Integer> frequency = new HashMap<>();
        for (int value : arr) {
            frequency.merge(value, 1, Integer::sum);
        }

        // Collect keys with count > 1
        Set<Integer> duplicates = new HashSet<>();
        for (Map.Entry<Integer, Integer> entry : frequency.entrySet()) {
            if (entry.getValue() > 1) {
                duplicates.add(entry.getKey());
```

```
            }
        }

        return Collections.unmodifiableSet(duplicates);
    }

    /**
     * Overload that works with any {@link Iterable} of {@link Integer}.
     *
     * @param numbers the numbers to inspect; may be {@code null}
     * @return an immutable set of duplicated values; never {@code null}
     */
    public static Set<Integer> findDuplicateValues(Iterable<Integer> numbers) {
        if (numbers == null) {
            return Collections.emptySet();
        }

        Map<Integer, Integer> frequency = new HashMap<>();
        for (Integer n : numbers) {
            if (n == null) {
                continue; // skip null elements
            }
            frequency.merge(n, 1, Integer::sum);
        }

        Set<Integer> duplicates = new HashSet<>();
        for (Map.Entry<Integer, Integer> e : frequency.entrySet()) {
            if (e.getValue() > 1) {
                duplicates.add(e.getKey());
            }
        }
        return Collections.unmodifiableSet(duplicates);
    }
}
```

Why this version is better

Criterion	Before	After
Time complexity	O(n²)	O(n)
Space usage	O(k) where *k* = number of duplicate *pairs*	O(u) where *u* = number of distinct values (optimal for this problem)

Result semantics	May contain many repeated entries	Guarantees each duplicate appears exactly once
Readability	Nested loops, no documentation	Self-documenting method name, Javadoc, clear variable names
Safety	No null-check	Handles `null` and empty inputs gracefully
Extensibility	Fixed to `int[]`	Overload for any `Iterable<Integer>`; can be further generalized to other numeric types.

---

5. New insights / best practices

1. **Prefer a single-pass counting approach** for duplicate detection.
   - `HashMap` (or primitive-specialized maps) gives amortized O(1) inserts/look-ups.
   - For very large data sets where memory is a concern, consider a **Bloom filter** for a probabilistic duplicate check, or external sorting.

2. **Return immutable collections** (`Collections.unmodifiableSet`) when you don't expect callers to modify the result. This prevents accidental side-effects.

3. **Document the contract** (null handling, mutability, order) with Javadoc. IDEs surface this information automatically.

4. **Avoid exposing internal mutable structures** (e.g., returning the raw `ArrayList` you built). It couples the implementation to the caller.

5. **Use `merge` / `compute`** methods of `Map` for concise counting logic; they reduce boilerplate and are less error-prone.

6. **Consider streaming APIs** if you need a one-liner or want to chain further operations:
   ```java
   Set<Integer> dup = Arrays.stream(arr)
                           .boxed()
                           .collect(Collectors.groupingBy(Function.identity(),
                                                   Collectors.counting()))
                           .entrySet()
                           .stream()
                           .filter(e -> e.getValue() > 1)
                           .map(Map.Entry::getKey)
                           .collect(Collectors.toUnmodifiableSet());
   ```

7. **If you truly need the *pairwise* duplicate list** (as the original code produced), keep the O(n²) algorithm but guard against adding the same value multiple times:

```java
Set<Integer> seen = new HashSet<>();
for (int i = 0; i < arr.length; i++) {
    for (int j = i + 1; j < arr.length; j++) {
        if (arr[i] == arr[j] && seen.add(arr[i])) {
            duplicates.add(arr[i]); // now added only once
        }
    }
}
```

8. **Testing** – write unit tests covering:
   - Empty / null input
   - No duplicates
   - One duplicate value appearing many times
   - Multiple distinct duplicate values
   - Negative numbers and zero (if relevant)

---

6. Follow-up questions

1. **Result expectation** – Do you need the method to return *unique* duplicate values (as in the refactor) or the *full list of matching pairs* as the original code does?
2. **Input source** – Is the data always an `int[]`, or do you sometimes work with `List<Integer>` / streams?
3. **Performance constraints** – Approximate size of the arrays you expect to process (e.g., millions of elements)? This influences whether a plain `HashMap` is sufficient or if a more memory-efficient solution is required.
4. **Thread-safety** – Will this method ever be called concurrently on shared data structures? If so, we might discuss immutable data handling or concurrent maps.

Feel free to let me know which variant you prefer, or share additional constraints so I can tailor the solution further.